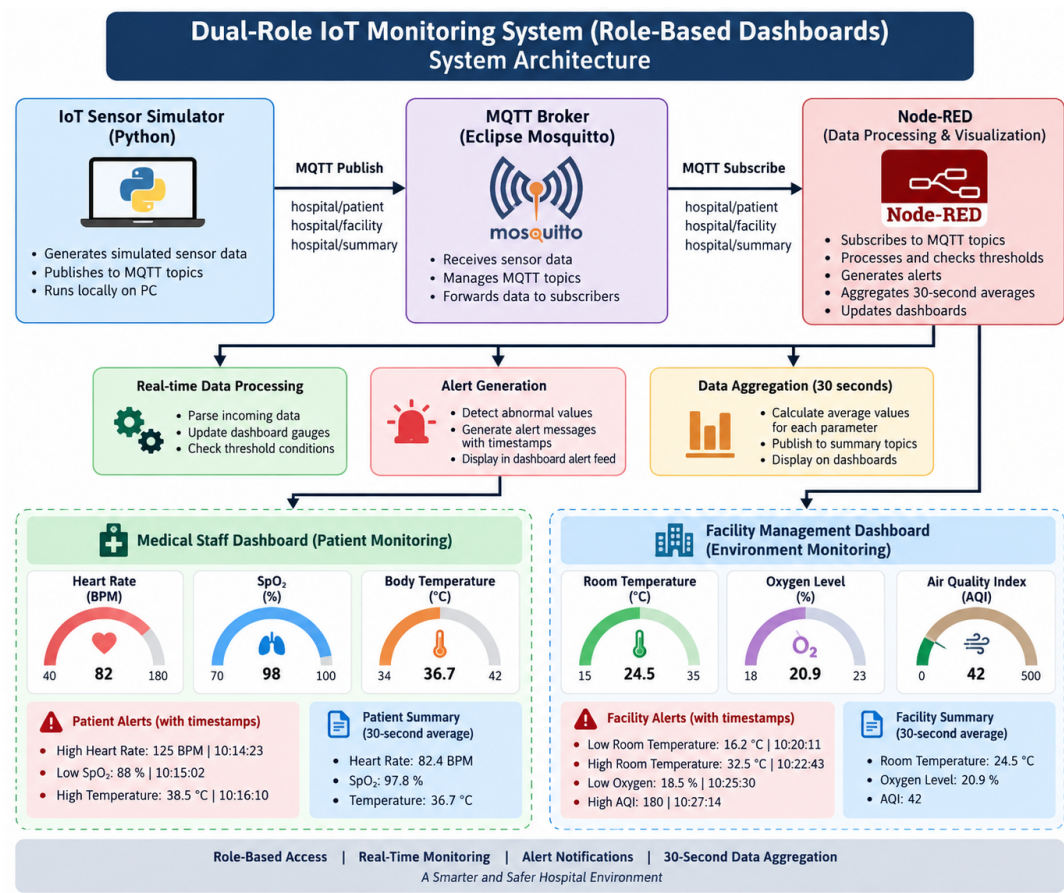


Dual-Role IoT Monitoring System with Role-Based Dashboards

ESP32 + MQTT + Node-RED + Wokwi



System architecture overview

Project Type	IoT Monitoring and Dashboard System
Prototype	ESP32 simulated in Wokwi
Communication	MQTT (Message Queuing Telemetry Transport)
Visualization	Node-RED / FlowFuse Dashboard
Prepared for	Internship Documentation

1. Abstract

This project presents a dual-role Internet of Things (IoT) monitoring system for a hospital environment. The system separates monitoring into two logical dashboards: a Medical Staff Dashboard for patient vital parameters and a Facility Management Dashboard for environmental conditions. An ESP32 prototype is simulated in Wokwi and uses Wi-Fi to publish sensor data through MQTT. Node-RED receives the data, evaluates predefined thresholds, generates timestamped alerts, provides real-time gauges, and performs 30-second data aggregation. The completed prototype was tested end-to-end and demonstrated live sensor updates, abnormal-condition alerts, persistent alert feeds, and summarized readings.

2. Introduction

Hospitals handle multiple categories of real-time information. Patient vital signs are primarily relevant to medical staff, while room temperature, oxygen concentration, and air quality support facility management. A role-oriented dashboard structure improves clarity by presenting each group with the information relevant to its responsibilities. This project implements that concept using hierarchical MQTT topics and separate dashboard groups while retaining a common Node-RED processing layer.

3. Objectives

- Develop an ESP32-based IoT data acquisition prototype using Wokwi.
- Monitor Heart Rate, SpO₂ (blood oxygen saturation), and Body Temperature.
- Monitor Room Temperature, Oxygen Level, and AQI (Air Quality Index).
- Use separate MQTT topics for patient and facility datasets.
- Display live values using dashboard gauges.
- Generate abnormal-condition alerts and retain recent alerts.
- Add timestamps to alerts and produce 30-second average summaries.
- Demonstrate a modular architecture that can later use physical sensors.

4. System Architecture

The final prototype uses an ESP32 simulated in Wokwi as the primary data source. Five potentiometers provide simulated analog inputs and a DHT22 provides room-temperature data. The ESP32 connects to Wi-Fi and publishes measurements to an MQTT broker. Node-RED subscribes to the topics and routes data to dashboards, alert processing, and aggregation functions.

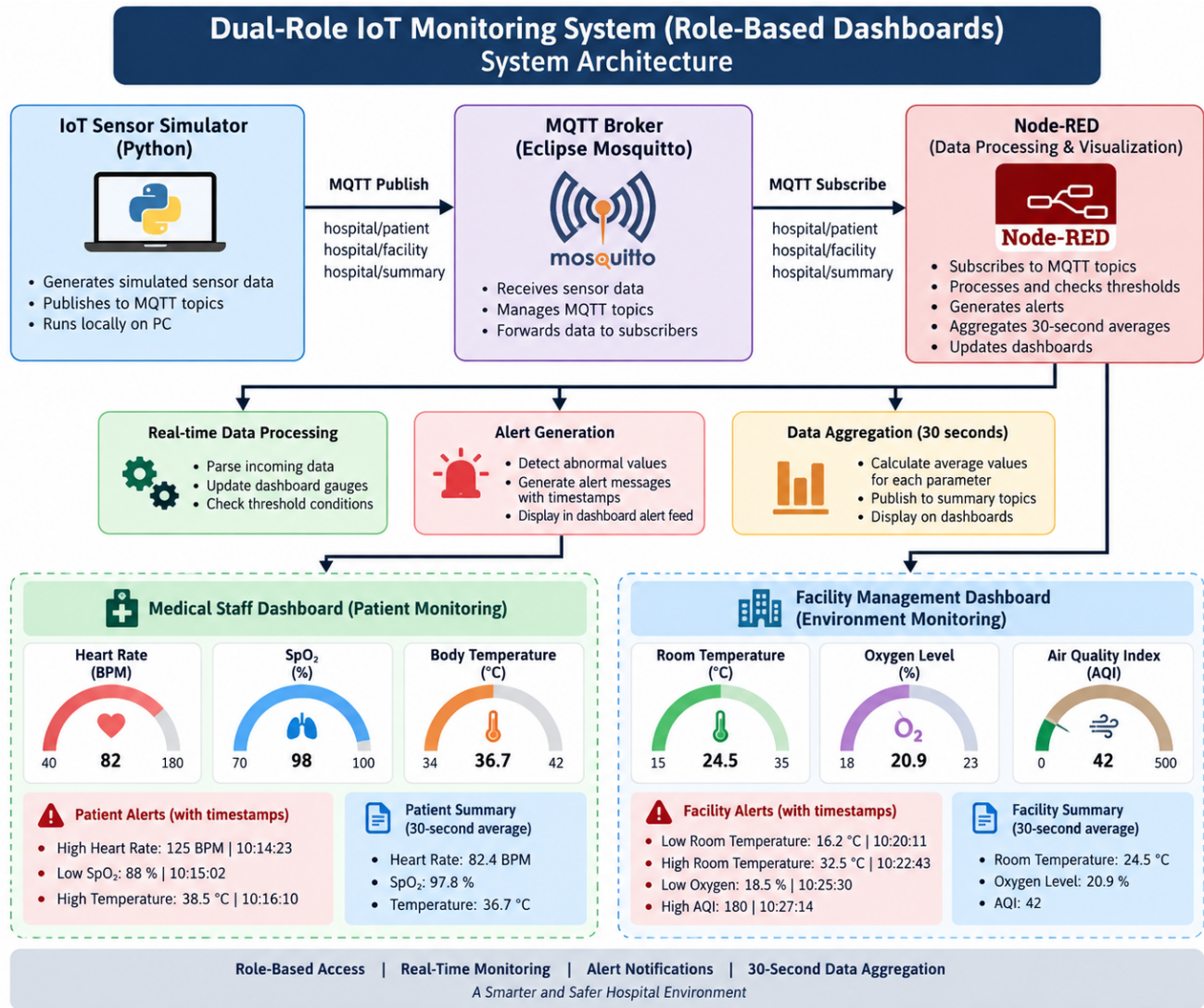


Figure 1. Overall system architecture.

Primary data path: Wokwi ESP32 → Wi-Fi → HiveMQ MQTT Broker → Node-RED → Dashboards / Alerts / Aggregation. During development, a Python simulator and local Mosquitto broker were also used to validate the MQTT and Node-RED pipeline before the ESP32 prototype was integrated.

5. Components and Technologies

Component / Technology	Purpose
ESP32 DevKit	Main controller, sensor acquisition, Wi-Fi and MQTT communication
5 × Potentiometers	Simulated Heart Rate, SpO ₂ , Body Temperature, Oxygen Level and AQI inputs
DHT22	Room-temperature measurement
Wokwi	Virtual ESP32 and sensor simulation
HiveMQ	MQTT broker used for the Wokwi prototype

Component / Technology	Purpose
Mosquitto	Local MQTT broker used during initial software validation
Node-RED / FlowFuse Dashboard	Processing, visualization, alerts and summaries
Python	Initial IoT simulator for pipeline testing

6. ESP32 Pin Configuration

The final Wokwi circuit uses ADC-capable ESP32 inputs for the potentiometer-based simulated sensors and a digital GPIO for the DHT22.

Input	Parameter	ESP32 Pin	Interface
Potentiometer 1	Heart Rate	GPIO 34	Analog / ADC
Potentiometer 2	SpO ₂	GPIO 35	Analog / ADC
Potentiometer 3	Body Temperature	GPIO 32	Analog / ADC
Potentiometer 4	Oxygen Level	GPIO 33	Analog / ADC
Potentiometer 5	AQI	GPIO 36 / VP	Analog / ADC
DHT22 DATA	Room Temperature	GPIO 4	Digital

Potentiometer wiring

- VCC → 3.3 V
 - GND → GND
 - SIG → assigned ESP32 GPIO
- DHT22 wiring: VCC → 3.3 V, GND → GND, DATA → GPIO 4.

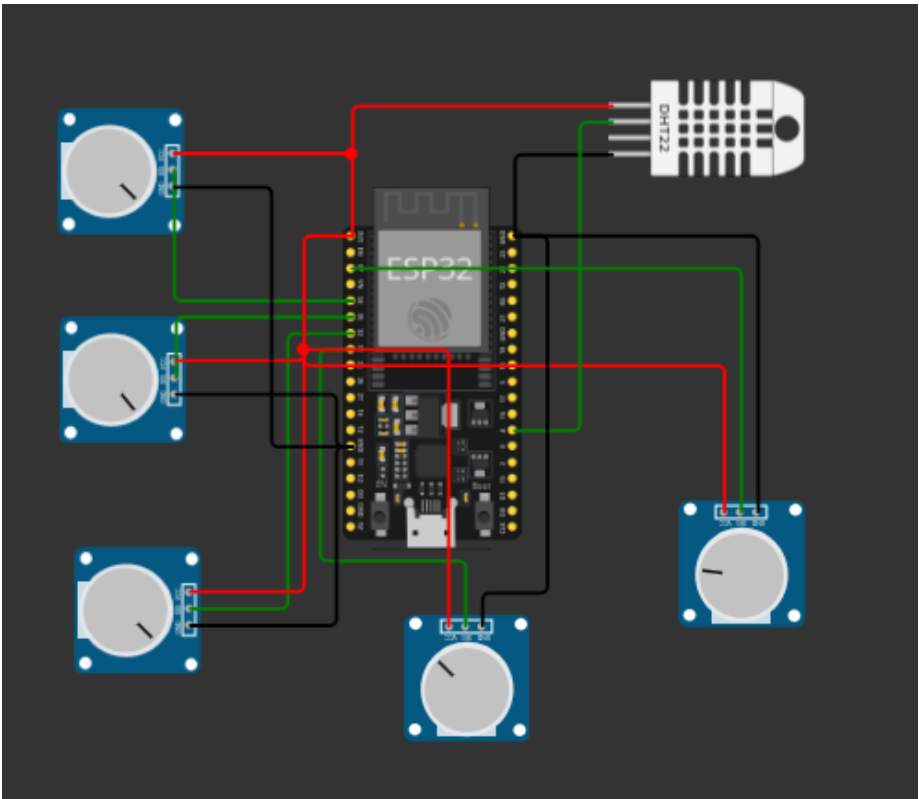


Figure 2. Wokwi ESP32 circuit used for the prototype.

7. MQTT Topic Structure

MQTT provides lightweight publish/subscribe communication. A hierarchical topic structure is used to separate patient and facility datasets and to provide dedicated topics for periodic summaries.

Dataset	MQTT Topic
Patient – Heart Rate	hospital/patient/heart_rate
Patient – SpO ₂	hospital/patient/spo2
Patient – Body Temperature	hospital/patient/body_temperature
Facility – Room Temperature	hospital/facility/room_temperature
Facility – Oxygen Level	hospital/facility/oxygen_level
Facility – AQI	hospital/facility/aqi
Patient Summary	hospital/summary/patient
Facility Summary	hospital/summary/facility

Role-based logical separation

- Medical Staff Dashboard subscribes to patient topics.
- Facility Management Dashboard subscribes to facility topics.
- Summary topics carry periodic aggregated information.

Production note: The prototype demonstrates logical role separation through topic hierarchy and dashboard subscriptions. A production hospital deployment should additionally use authentication, access-control lists (ACLs), Transport Layer Security (TLS), encryption, and secure device provisioning.

8. Medical Staff Dashboard

The Medical Staff Dashboard provides real-time visibility of patient vital parameters. Gauge widgets display current values, while the persistent alert feed records recent abnormal events and timestamps.

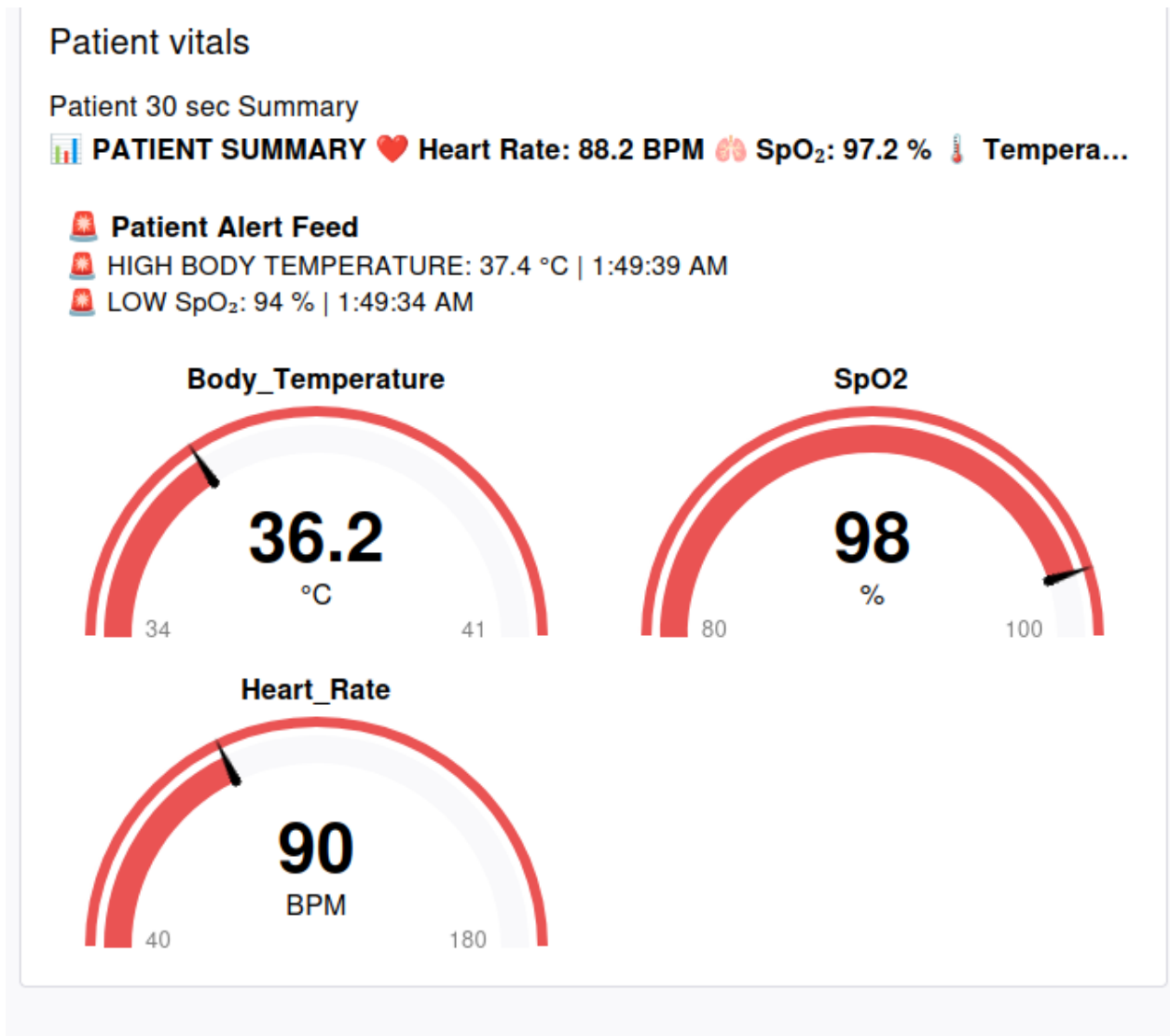


Figure 3. Medical Staff Dashboard.

- Heart Rate, SpO₂ and Body Temperature gauges.
- Patient Alert Feed with recent timestamped events.
- Patient 30-second summary with averaged readings.

9. Facility Management Dashboard

The Facility Management Dashboard focuses on environmental conditions. It provides separate gauges, a persistent alert feed, and a periodic summary for facility-related parameters.

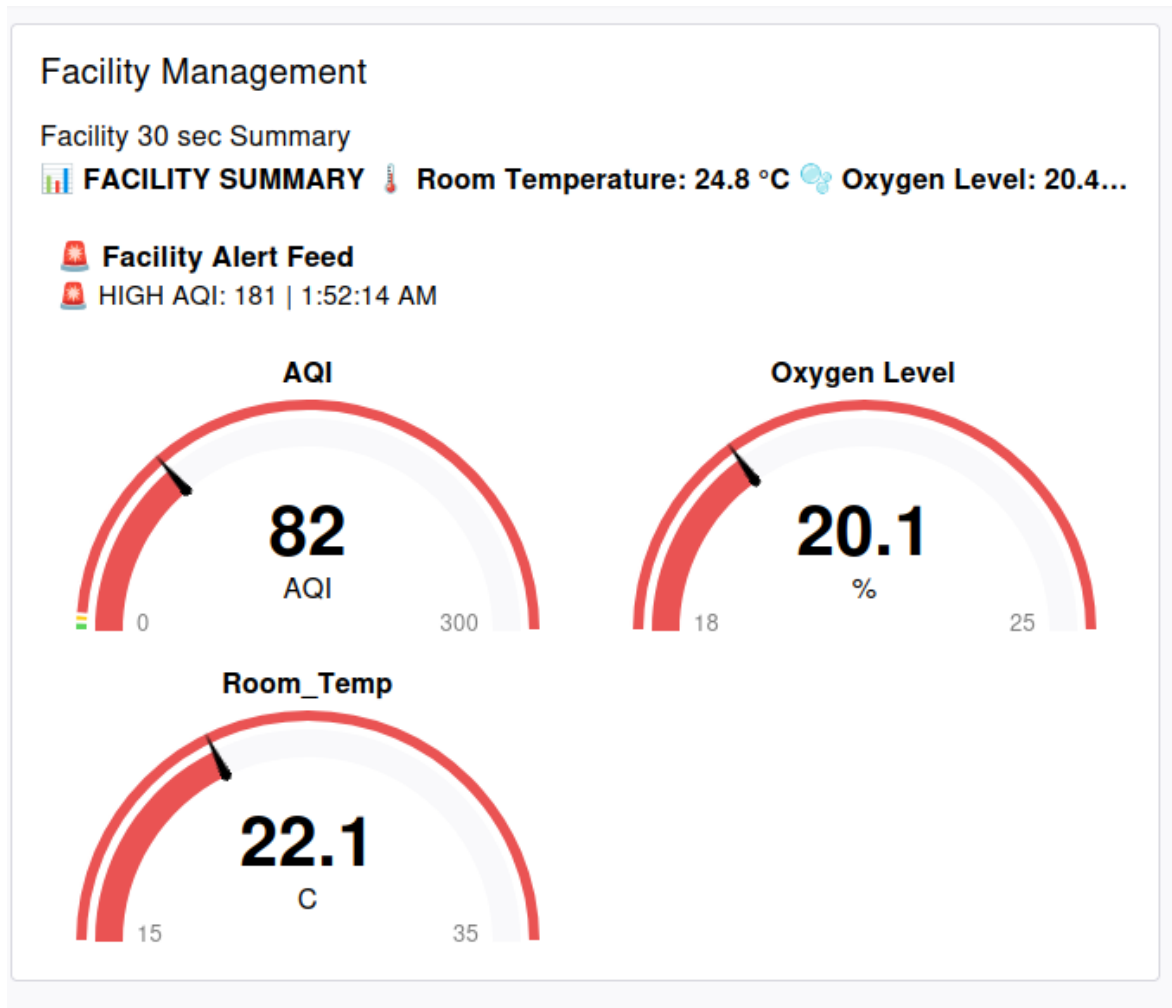


Figure 4. Facility Management Dashboard.

- Room Temperature, Oxygen Level and AQI gauges.
- Facility Alert Feed with timestamped abnormal events.
- Facility 30-second summary with averaged readings.

10. Alert Generation

Node-RED Switch and Function nodes compare incoming measurements against predefined prototype thresholds. When a threshold is crossed, a formatted alert is generated and stored in a persistent feed. The feed retains up to ten recent alerts, with the newest alert first.

Parameter	Prototype alert condition
Heart Rate	Low < 60 BPM; High > 100 BPM
SpO ₂	Low < 95 %
Body Temperature	Low < 36.1 °C; High > 37.2 °C
Room Temperature	Low < 20 °C; High > 26 °C
Oxygen Level	Low < 19.5 %; High > 23.5 %
AQI	High > 50

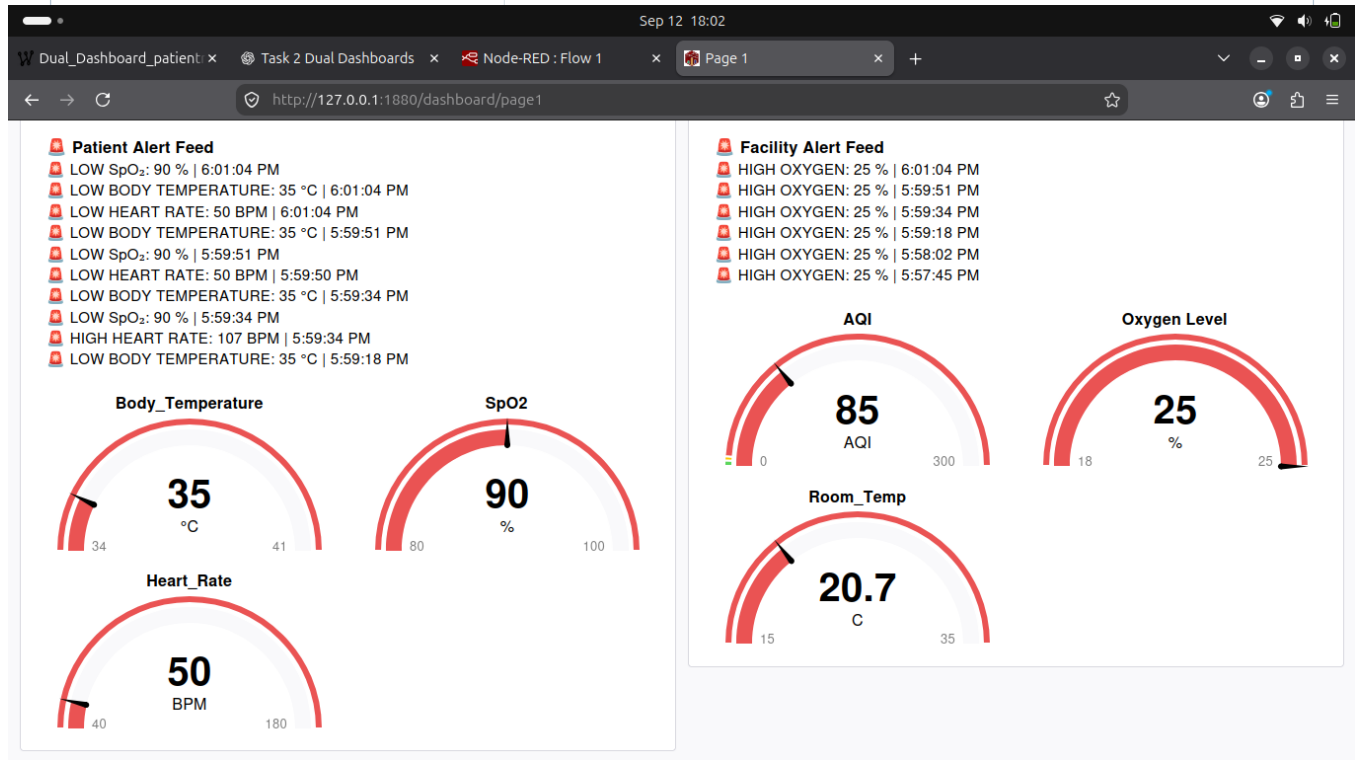


Figure 5. Timestamped patient and facility alerts with live gauge values.

These thresholds are demonstration values selected for prototype testing and should not be interpreted as clinical diagnostic limits.

11. Periodic Data Aggregation

Incoming measurements are stored in Node-RED context memory for a 30-second interval. At the end of the interval, the system calculates the average for each parameter, publishes the result to a summary MQTT topic, and formats the result for dashboard display.

Patient summary

- Average Heart Rate
- Average SpO₂
- Average Body Temperature

Facility summary

- Average Room Temperature
- Average Oxygen Level
- Average AQI

```
----- PATIENT DATA -----
Heart Rate      : 107 BPM
SpO2            : 90 %
Body Temperature : 35.0 °C

----- FACILITY DATA -----
Room Temperature : 20.7 °C
Oxygen Level     : 25.0 %
AQI              : 85
-----

----- PATIENT DATA -----
Heart Rate      : 50 BPM
SpO2            : 90 %
Body Temperature : 35.0 °C

----- FACILITY DATA -----
Room Temperature : 20.7 °C
Oxygen Level     : 25.0 %
AQI              : 85
```

Figure 6. ESP32 Serial Monitor showing live patient and facility readings.

12. Implementation Workflow

ESP32 firmware

- Connects to Wokwi-GUEST Wi-Fi.
- Uses PubSubClient for MQTT communication.
- Reads five analog inputs and one DHT22 temperature input.
- Converts analog values into realistic prototype ranges.
- Publishes six measurements every five seconds.

Node-RED processing

- MQTT-in nodes receive the sensor streams.
- Gauge nodes provide real-time visualization.
- Switch and Function nodes implement threshold alerts.
- Alert-feed functions retain recent timestamped events.
- Aggregation functions calculate 30-second averages.

Testing approach

A Python simulator with local Mosquitto was first used to validate the MQTT and Node-RED pipeline. The data source was then replaced by the Wokwi ESP32 prototype, while the topic hierarchy and dashboard processing were retained.

13. Testing and Results

Test	Result
ESP32 Wi-Fi connection	Successful
MQTT broker connection	Successful
Six live sensor values	Successful
Patient threshold alerts	Successful
Facility threshold alerts	Successful
Timestamped alert feeds	Successful
Patient 30-second summary	Successful
Facility 30-second summary	Successful
Node-RED dashboard updates	Successful

Observed result

The complete prototype demonstrated end-to-end sensor acquisition, MQTT communication, real-time dashboard visualization, abnormal-condition alerting, persistent alert feeds, and periodic summary generation. The supplied screenshots provide evidence from the Wokwi circuit, ESP32 Serial Monitor, and Node-RED dashboards.

14. Advantages

- Separates patient and facility information according to user role.
- Provides real-time monitoring through MQTT and Node-RED.
- Detects abnormal conditions automatically.
- Maintains recent alerts rather than only the latest event.
- Provides periodic summaries for easier interpretation.
- Uses a modular topic structure that can be expanded with additional sensors.

15. Limitations

- Sensor inputs are simulated in Wokwi; physical medical-grade sensors are not used in this prototype.
- The public MQTT broker is suitable for demonstration/testing, not sensitive production hospital data.
- Prototype thresholds are for demonstration and are not clinical limits.
- Production deployment requires secure authentication, authorization, encryption and privacy controls.

16. Future Enhancements

- Replace Wokwi inputs with physical ESP32-compatible sensors.
- Use authenticated MQTT with role-specific access-control lists.
- Enable TLS encryption and secure device provisioning.
- Store historical measurements in a time-series database.
- Add trend graphs, long-term analytics and automated reports.
- Deploy multiple ESP32 nodes for different hospital rooms.

17. Conclusion

The Dual-Role IoT Monitoring System successfully demonstrates how an IoT architecture can separate hospital patient monitoring from facility monitoring while maintaining a common communication and processing layer. The Wokwi ESP32 prototype provides sensor data, MQTT provides lightweight publish/subscribe communication, and Node-RED provides visualization, threshold checking, alert management and periodic aggregation. The completed implementation therefore addresses the core project requirements of role-based dashboards, separate MQTT datasets, gauge-based monitoring, abnormal-condition alerts, and summarized insights.

18. Technologies Used

Technology	Use
ESP32	IoT controller and Wi-Fi connectivity
Wokwi	Virtual hardware and sensor simulation
Arduino C++	ESP32 firmware
PubSubClient	MQTT client library
DHT22	Room-temperature sensing
MQTT	Publish/subscribe communication
HiveMQ	MQTT broker for Wokwi prototype
Mosquitto	Local broker for initial validation
Node-RED / FlowFuse Dashboard	Processing and visualization
Python	Initial software-based IoT simulation

End of Report